Coding Challenge: Longest Substring with At Most K Distinct Characters

Problem Statement

Given a string `s` and an integer `k`, return the length of the longest substring of `s` that contains at most `k` distinct characters.

1. Standard Cases (Basic Validation)

- **Test Case 1:** General mix of repeating and distinct characters.
  - `s = "eceba"`, `k = 2`
  - *Expected Output:* `3` (Substring: `"ece"`)
- **Test Case 2:** k is larger than the total unique characters in the string.
  - `s = "abc"`, `k = 5`
  - *Expected Output:* `3` (Substring: `"abc"`)

2. Edge Cases (Boundaries & Constraints)

- **Test Case 3:** k = 0 (No distinct characters allowed).
  - `s = "abcdef"`, `k = 0`
  - *Expected Output:* `0`
- **Test Case 4:** String containing only one unique character.
  - `s = "aaaaaaa"`, `k = 1`
  - *Expected Output:* `7` (Substring: `"aaaaaaa"`)
- **Test Case 5:** String length is exactly 1.
  - `s = "a"`, `k = 1`
  - *Expected Output:* `1`

3. Advanced Patterns (Testing Window Shifting)

- **Test Case 6:** The valid substring is at the very end of the string.
  - `s = "abaccc"`, `k = 2`
  - *Expected Output:* `4` (Substring: `"accc"`)
- **Test Case 7:** The valid substring is in the exact middle.
  - `s = "abcdefghba"`, `k = 3`
  - *Expected Output:* `3` (Substrings like `"abc"`, `"bcd"`, etc.)
- **Test Case 8:** Characters alternate rapidly, forcing the left pointer to move frequently.
  - `s = "abababdbaba"`, `k = 2`
  - *Expected Output:* `6` (Substring: `"ababab"`)